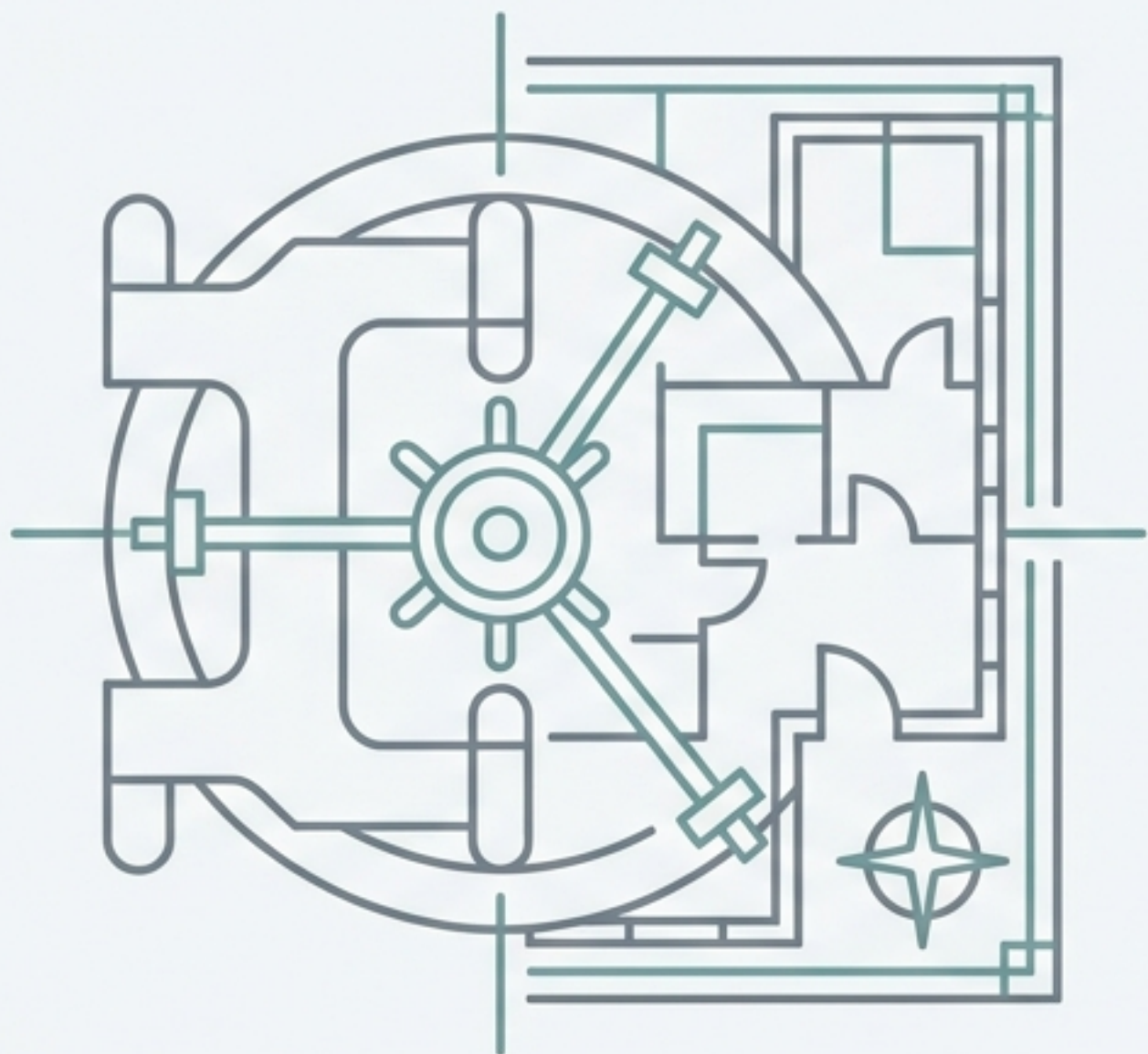




Aula 3: O Cofre e a Planta Baixa

Princípios de Orientação a Objetos em Java:
Encapsulamento, Construtores e Membros Estáticos.

O Perigo do Estado Inválido

```
Conta minhaConta = new Conta();  
minhaConta.saldo = -5800.00; // Acesso direto e sem restrições
```



Erro Crítico de Negócio:
**Conta com saldo negativo sem
ter limite pré-aprovado.**



Insight: Quando os atributos de um objeto estão expostos, não há como garantir as regras de negócio. O objeto perde sua integridade estrutural.

O Princípio do Encapsulamento



Ocultamento de Informação (Information Hiding)

Esconder como as rotinas internas funcionam.

Separação de Papéis

A interface (o que o objeto faz) é separada da implementação (como ele faz).

Analogia do Caixa Eletrônico

Um usuário interage apenas com os botões e a tela (Interface). O cofre e os circuitos de contagem de notas (Implementação) ficam isolados e protegidos contra manipulação direta.

A Matriz de Modificadores de Acesso

	Mesma Classe	Mesmo Pacote	Subclasse	Global
private	✓	✗	✗	✗
default (package-private)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

O padrão ouro para atributos.

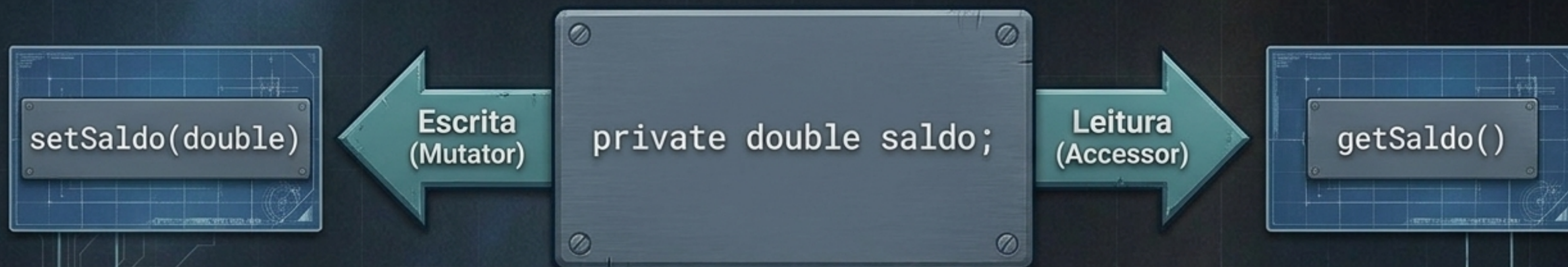
Ausência de modificador.

Focado em Herança.

O padrão para a interface.

[Em um design seguro, os atributos nascem private e apenas os métodos essenciais são expostos como public.]

Os Guardiões do Estado: Getters e Setters



A Convenção Java: O acesso controlado é feito prefixando os métodos com `get` (para retornar o valor) e `set` (para alterar o valor).

Controle de Leitura/Escrita:

- Um atributo apenas com Getter é *Read-Only* (Somente leitura).
- Um atributo sem métodos de acesso é estritamente de uso interno.

⚠ Atenção: Nunca gere Getters e Setters cegamente. Só crie os métodos que a regra de negócio realmente exige.

Protegendo as Regras de Negócio

Código Frágil

```
conta.saldo = -500;  
// Aceita qualquer valor
```

Código Seguro

```
public void deposita(double valor) {  
    if (valor > 0) {  
        this.saldo += valor;  
    } else {  
        // Lança erro ou ignora  
    }  
}
```

[O uso da palavra-chave **this** garante que estamos alterando o atributo da instância atual, diferenciando-o do parâmetro do método.]

O Nascimento do Objeto: Construtores



Operador

Objeto Válido em Memória

O que é? Uma rotina especial chamada automaticamente pelo operador `new` no exato momento em que o objeto é criado em memória.

Características Físicas:

- Tem exatamente o mesmo nome da classe.
- Não possui tipo de retorno (nem mesmo `void`).

Propósito: Garantir que o objeto nasça em um estado válido, com seus atributos essenciais já preenchidos.

Construtor Padrão vs. Construtor Personalizado

O Construtor Invisível (Default)

```
public Conta() {}
```

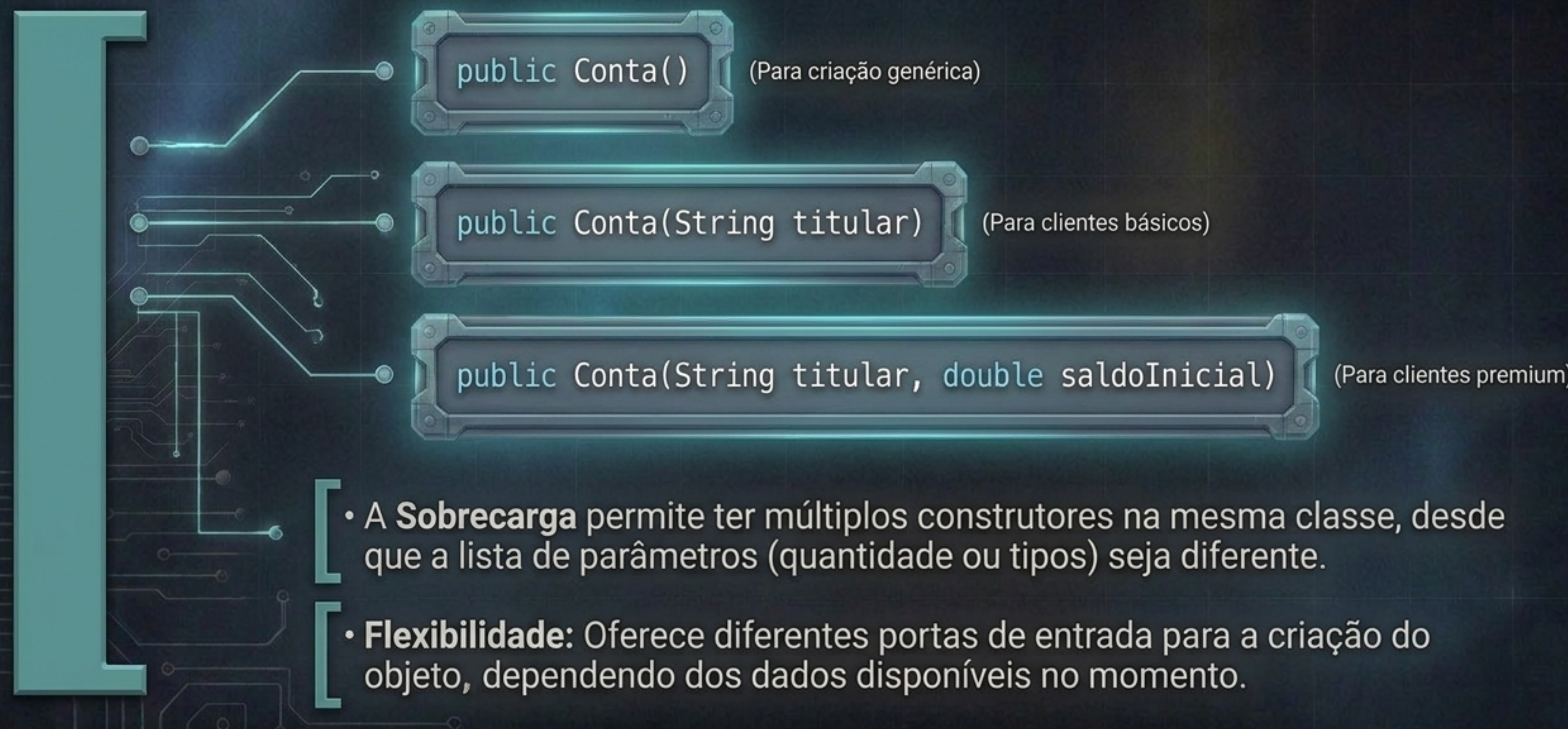
Se nenhum construtor for escrito, o Java fornece um vazio. Atributos nascem com valores zerados/nulos.

A Imposição de Regras (Custom)

```
public Conta(String titularCPF) {  
    this.cpf = titularCPF;  
}
```

A partir do momento em que você escreve qualquer construtor personalizado, o construtor padrão desaparece. A passagem do CPF torna-se obrigatória para o nascimento da Conta.

Sobrecarga (Overloading) de Construtores



```
public Conta()
```

(Para criação genérica)

```
public Conta(String titular)
```

(Para clientes básicos)

```
public Conta(String titular, double saldoInicial)
```

(Para clientes premium)

- A **Sobrecarga** permite ter múltiplos construtores na mesma classe, desde que a lista de parâmetros (quantidade ou tipos) seja diferente.
- **Flexibilidade:** Oferece diferentes portas de entrada para a criação do objeto, dependendo dos dados disponíveis no momento.

Princípio DRY: Reuso com this(...)

```
public Conta(String nome) {  
    this(nome, "000.000.000-00", 1);  
}
```



- Evite duplicar a lógica de inicialização em múltiplos construtores.
- **Regra de Ouro:** A chamada `this(...)` deve ser obrigatoriamente a primeira linha executada dentro do bloco do construtor.

Delega para o construtor principal

Estado do Objeto vs. Estado da Classe



Variáveis de Instância:

Pertencem ao objeto. Cada vez que usamos new, uma nova cópia é criada (ex: saldo, titular).

Membros Estáticos (static):

Pertencem à Classe. Existe apenas uma única cópia na memória, compartilhada por todos os objetos daquela classe.

O Modificador **static** na Prática

```
private static int totalDeContas = 0;
```

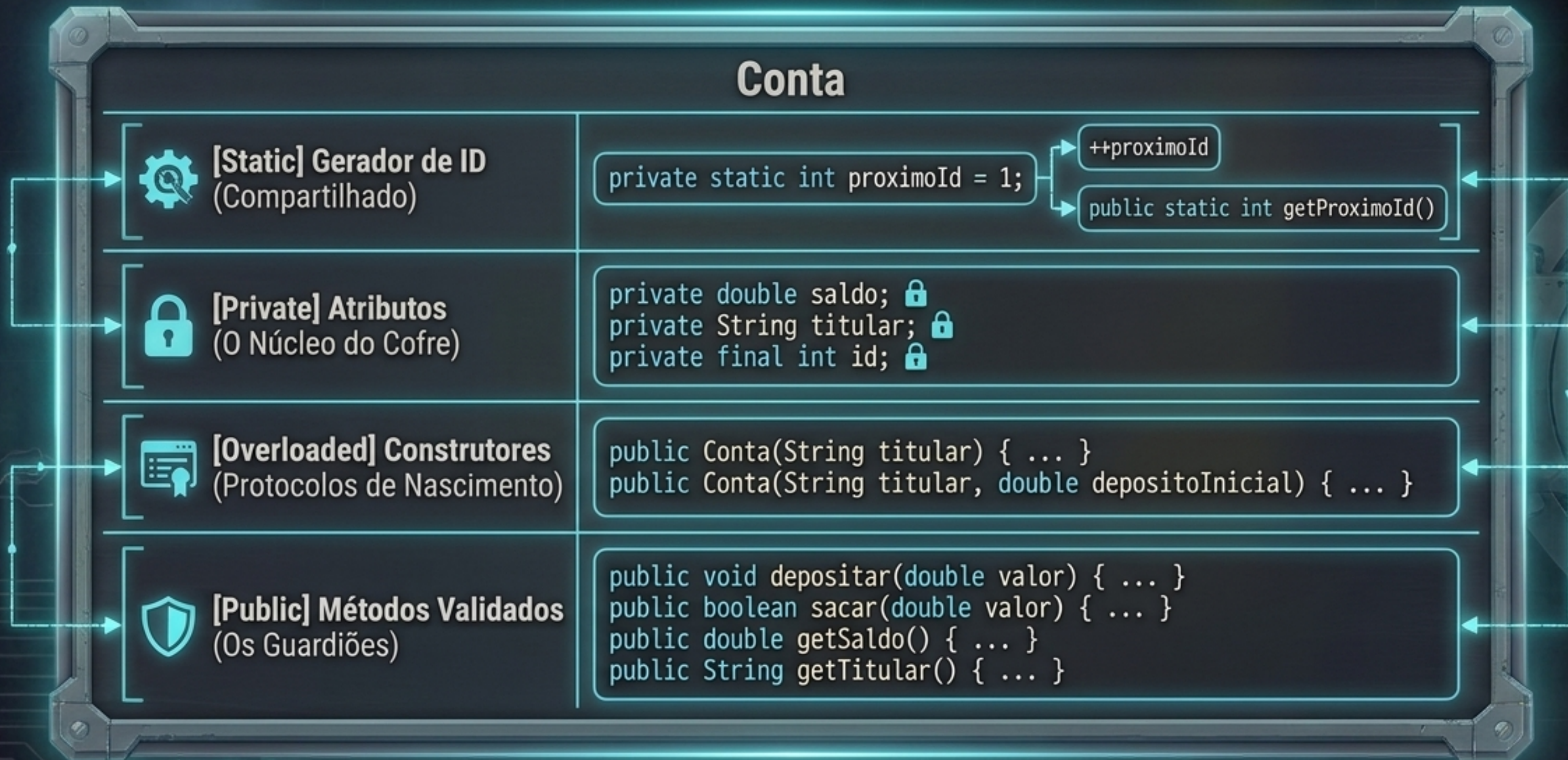
Dentro do construtor:
`Conta.totalDeContas++;`

```
public static int getTotalDeContas() {  
    return Conta.totalDeContas;  
}
```

• **Contadores Globais:** Ideal para gerar IDs únicos ou rastrear o número total de instâncias criadas.

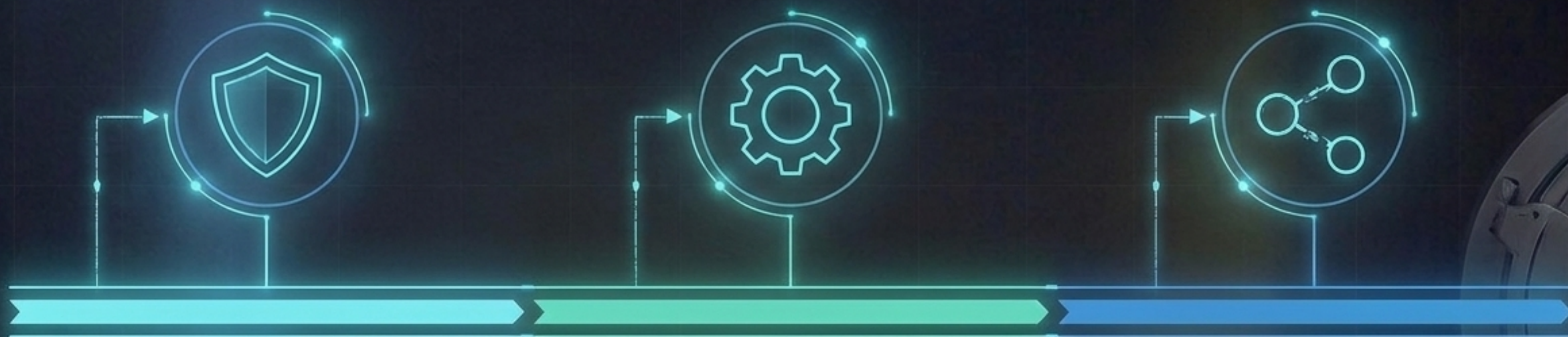
Regra de Acesso: Métodos estáticos não podem usar a palavra-chave `this`, pois não sobre uma instância específica. O acesso é feito via `NomeDaClasse.membro`.

Síntese: O Objeto Seguro e Robusto



A convergência de **Ocultamento de Informação**, **Inicialização Obrigatória** e **Controle de Estado Global**. O objeto agora é uma **estrutura autossuficiente e inviolável**.

Laboratório Prático (Hands-On)



Duração: 1 hora e 40 minutos.

Objetivo: Transformar as classes vulneráveis **Conta** e **Funcionario** das aulas anteriores em estruturas robustas utilizando o conceito de Defensive Programming.

Dinâmica: Aplicação direta de Modificadores, Construtores e Estáticos com validação de regras de negócio reais.

Roteiro de Exercícios



Exercício 1 - A Blindagem

- Tornar todos os atributos de Conta/Funcionario private.
- Criar Getters/Setters seletivos.
 - Implementar regras de validação nos métodos de saque, depósito e salário.



Exercício 2 - O Nascimento

- Criar um construtor obrigatório forçando a passagem do titular e CPF.
- Criar um construtor alternativo (Sobrecarga).
- Utilizar this(...) para reaproveitar código.



Exercício 3 - A Coletividade

- Criar um atributo static de contagem de instâncias.
- Gerar um Identificador Único (ID) automático para cada novo objeto instanciado.